I have a task to implement a webapp working prototype. I'll use OpenSpec framework and approach. Let's discuss the task first and prepare it to the implementation. I expect out of this conversation the following files to be created: task.md with the task itself (unmodified), requirements.md - the list of functional and non-functional requirements, main scenarios, what we put out of scope, assumptions, architecture.md - high level overview of main parts and components of the system, how do they communicate, essential architectural decisions and technologies used, backlog.md - the implementation plan step-by-step with the tasks in 1-2 phrases, those will become the OpenSpec proposals. Let's start. Here is the task itself "Let's imagine we're working at a company that already has clients and a product. We're
growing well. We have a backend with business logic and clients who interact with it through
an API.
Right now, all clients — regardless of their size — can use our API the same way. We want to
change that. We want to introduce tiers:
● Gold tier: Can make a very large number of API calls per day (e.g., 1 million)
● Silver tier: Moderate number
● Bronze tier: A smaller number (e.g., a few hundred per day)
The exact numbers aren't important right now.
Build a working prototype of three things:
1. The backend. Enforces a per-tenant daily quota by tier. The limit resets at midnight (no
sliding windows). If a client hits the limit, return an HTTP 4xx error indicating the limit is
reached.
2. An admin dashboard (React). It should let an operator answer, at a glance and in real
time: how much traffic are we taking right now, which tenants are close to or over their
limit, and how many requests are we rejecting. We expect some kind of live dashboard
that gives the admin the information about service state, tenants, their limits and
possibility to change tier per tenant. Handle the unhappy states: still loading, backend
down, connection dropped and recovered. We will not judge you on visual design, and
you do not need a component library or a design system — but we should be able to
read the screen.

3. A way to generate or simulate traffic, so the dashboard visibly moves when we run it.
The following describes the expected production behavior and should inform your design
decisions. The prototype itself does not need to fully implement or reproduce these
constraints and may use reasonable simplifications where necessary:
● There will be an allowed number of API calls per client (tenant) per day
● The limit resets at midnight (no sliding windows)
● If a client hits the limit, return an HTTP 4xx error indicating the limit is reached
● ~250,000 requests/minute at peak; ~100,000 req/min at off-peak. ~50,000 tenants
total"

Show more

Recalled 2 memories ▸

Good task for SDD: the core rule is small, but there are several design decisions that will shape every proposal later. Here's how I suggest we go through it, one decision at a time, then I'll write the four files at the end (task.md verbatim, plus requirements, architecture, backlog).

Rough agenda

1. Stack and runtime (what we build with, how it starts)
2. The "business API": what endpoint the quota protects, how a tenant is identified
3. Quota semantics: which timezone "midnight" is, whether rejected calls count, response format of the 4xx
4. Counting under load: the numbers translate to about 4,200 req/s at peak (250,000 / 60) and about 1,700 req/s off-peak, across 50,000 tenants, so where counters live and how tier lookups stay cheap matter
5. Tier changes: when they take effect and what happens to today's usage
6. Dashboard data: what metrics, how they get to the browser live (polling, SSE, WebSocket), and how to show "close to limit" when there are 50,000 tenants
7. Unhappy states: loading, backend down, dropped and recovered connection
8. Traffic generator: separate process vs. controls in the dashboard, how configurable
9. Out of scope and assumptions (admin auth, multi-region, billing, etc.)

First question: stack. Your earlier OpenSpec trial used Java 21 + Spring Boot (Gradle Kotlin DSL), React + TypeScript (Vite), Redis, docker compose with a bootstrap script, and JUnit + Testcontainers / Vitest + RTL for tests. Do you want the same stack and the same "one command starts everything" setup here, or is anything different for this project?

Yes, let's use this exact stack - java21 + Spring Boot, React + TypeScript, Redis as counters storage, docker and docker compose to wire everything up. Let's not implement database and message broker for the moment, we'll store everything in memory to simplify the prototype implementation.

Weighing whether multiple backend instances would break the setup ▸

Settled: Java 21 + Spring Boot, React + TypeScript, Redis for counters, docker compose wiring, no database or message broker.

One consequence I'll record as an assumption: tenants and their tiers live in the backend's memory (seeded at startup), and only counters live in Redis. That's consistent as long as we run a single backend instance; with several instances, a tier change made on one wouldn't reach the others. For the prototype I'd state "single backend instance; tenant registry moves to a database in production" and move on, unless you want tiers in Redis too.

Question 2: the protected API and tenant identity.

My proposal:

- One mock business endpoint, e.g. `GET /api/v1/resource`, that just returns a small payload. It stands in for "the existing business logic."
- The tenant is identified by an API key in a header (`X-API-Key`). The backend maps key → tenant → tier. Unknown or missing key returns 401 and is not counted against anyone.
- Quota enforcement runs as a servlet filter in the same Spring Boot app, in front of the business controllers, rather than as a separate gateway service. Fewer moving parts for a prototype, and in architecture.md we note that in production this would naturally move to an API gateway or sidecar.

Does that match what you had in mind, or would you prefer a simpler tenant header (e.g. `X-Tenant-Id`, no keys) or a separate gateway service?

ideally it is planned to be a standalone gateway service. If we implement it this way, what complexity it adds? Do I need to implement the mock backend? I think now, just mock response

Comparing the cost of a proxying gateway setup ▸

A real standalone gateway that proxies to a separate backend adds roughly this:

- **One more service** in compose: the gateway plus a mock upstream, with routing config between them.

- **Proxying concerns:** forwarding headers and bodies, timeouts, and mapping upstream failures to 502/504. Strictly, this also means deciding whether a request counts against quota when the upstream fails.
- **Framework choice:** Spring Cloud Gateway is reactive (WebFlux) by default, which means reactive Redis access and a different filter and testing model. There is a servlet-based variant (Gateway Server MVC) that avoids this, but it's still an extra dependency to learn.
- **An extra network hop** per request, so latency is measured across two services.

None of it is hard. It just doesn't show anything about quotas, tiers or the dashboard, which is what the task is judged on.

So I agree with "just mock response," and I'd shape it like this. We build a **standalone gateway service** whose upstream is a stub: the quota filter runs first, then a stub handler returns a fixed mock payload where a proxy route would later sit. Architecturally it's the gateway, and the stub is clearly marked as the replacement point for a real upstream. No separate mock backend container, and plain Spring MVC with a servlet filter, with no Spring Cloud Gateway for now. In architecture.md we record "upstream proxying is the next step; stub keeps the prototype focused."

That leaves one placement decision. **Where does the admin API (tenant list, usage, tier changes, live metrics stream) live?**

- **A. Inside the gateway service, under a separate path** (e.g. `/admin/**`, exempt from quota). It's simplest, because the tenant registry is in the gateway's memory, so tier changes apply instantly with no sync.
- **B. A separate admin service.** This is cleaner separation, but it needs a way to push tier changes to the gateway and read counters. With no database or broker, that means sharing tenant config through Redis (and pub/sub for changes).

I'd go with A for the prototype and describe B as the production direction. Which do you prefer? (I'm keeping `X-API-Key` for tenant identity unless you say otherwise.)

> Definitely A for the prototype with assumption mentioned that this would be the separate service. For the API key, let's assume the tenant is resolved on the layer before and here we already have the tenantId. So we take out of the scope the token expiration issue, unknown user and so one

Settling the status code for the tenant header ▸

Got it: the gateway receives an already-resolved tenant in a header (say `X-Tenant-Id`) from an upstream auth layer. Authentication, key management, token expiry and unknown tenants are out of scope. The only defensive behavior I'd keep is returning 400 if the header is missing, so the filter never crashes on bad input. Nothing more elaborate.

Question 3: quota semantics. I see three sub-decisions. My proposals:

1. **Which midnight.** A single global reset at midnight UTC for all tenants. Per-tenant timezones are a production consideration we can list as out of scope. With day-scoped counter keys (e.g. `quota:{tenantId}:{2026-11-05}`), the reset happens on its own when the date changes, with no reset job.
2. **What consumes quota.** Only allowed requests consume quota. Rejected requests don't eat into tomorrow or inflate usage, but we count them separately per tenant per day, because the dashboard needs "how many are we rejecting." So a tenant's day has `used` (at most the limit) and `rejected`.
3. **The 4xx response.**
   - 429 Too Many Requests, with a small JSON error body: tenantId, limit, used, reset time.
   - Headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` (epoch seconds of the next UTC midnight), and `Retry-After` (seconds until reset).
   - The limit and remaining headers also go on successful responses, so clients can see they're getting close.

Do these three hold, or do you want any of them different? For example, per-tenant timezones, or counting every attempt including rejected ones.

1. no, we don't need per tenant timezone or other individual activity. The date starts for everyone and all the quotes resets. What is required for the prototype implementation - the different interval of limit reset. Let's make it 1 minute. So we assume the day duration is 1 minute. 2. only allowed requests consumes quota. There is no "tomorrow requests" and borrowing activity. So, yes, the tenant's day has used and rejected. 3. The 429 response with repeat-after header. No need to put the current consumtion number in each response

Scaling the quota limits to match the short period ▸
Recorded:

- **Quota period is configurable and aligned for everyone.** Production is 1 day; the prototype runs with a period of 1 minute. Periods are aligned to UTC epoch time, so every tenant's quota resets at the same boundary. Counter keys become `quota:{tenantId}:{periodIndex}`, where `periodIndex = epochSeconds / periodSeconds`.
- **Tier limits are configured per period.** Gold's 1,000,000/day would never be hit in a 1-minute window by our generator, so prototype limits are sized so the dashboard visibly shows tenants approaching and crossing them, e.g. Bronze 100, Silver 1,000, Gold 10,000 per period.
- **Only allowed requests consume quota.** Each tenant's period has `used` and `rejected`.

- 429 with `Retry-After` (seconds to the next period boundary) and a minimal error body. No consumption headers on any response.

Question 4: counting under load and Redis failure.

At peak, 250,000 req/min ≈ 4,200 req/s, so about 4,200 counter operations per second against Redis. A single Redis instance handles on the order of 100,000 simple ops/s, so throughput isn't the problem. Correctness is.

Proposal: an atomic check-and-increment in a Lua script. A naive read, then compare, then increment lets two concurrent requests both see `used = 99` with a limit of 100 and both get through. The same happens across several gateway instances. A Lua script runs atomically in Redis and does the whole decision in one round trip: if `used < limit`, increment `used` and allow; otherwise increment `rejected` and deny. Both keys get a TTL of two periods so old periods clean themselves up. The tier limit is passed in from the gateway's in-memory tenant registry, so a tier lookup is a local map read.

The actual decision: what happens when Redis is unavailable?

- Fail-open: allow the request, log it, and expose a "quota enforcement degraded" metric that shows on the dashboard. Paying customers keep working; the cost is that some tenants may exceed their quota during the outage.
- Fail-closed: reject with 503. The quota stays strict, but a counter outage becomes a full API outage for every tenant.

I'd recommend fail-open, since the quota is a commercial limit rather than a security boundary, and an outage of the counting layer shouldn't take down the product. Fail-open or fail-closed?

yes, exactly all of these. Let's do fail-open

Checking how upgrades and downgrades take effect mid-period ▸

Fail-open it is: on Redis errors the gateway allows the request, logs it, and exposes a degraded-enforcement signal that the dashboard shows prominently.

Question 5: tier changes.

My proposal:

- Effect is immediate. The operator changes the tier via the admin API (e.g. `PUT /admin/tenants/{id}/tier`), the in-memory registry updates, and the very next request uses the new limit. A request already in flight may still be evaluated with the old limit; that's acceptable.
- Current period usage is preserved. We don't reset or trim counters on a tier change:

  - Upgrade: a Bronze tenant blocked at 100/100 who moves to Silver (1,000) resumes immediately with 900 remaining.
  - Downgrade: a Silver tenant at 500 who moves to Bronze (100) is rejected for the rest of the period, and `used` stays at 500, visibly over the new limit on the dashboard.
- **Only tier assignment is editable.** Tier definitions (the limit per tier) are static configuration. No audit log or history of changes in the prototype.

Does this match, or do you want different behavior on downgrade or editable tier limits?

let's implement the immediate change, but put in out of the scope - the current period usage preserved case as a preferable enhancment

Planning the dashboard's data and how it reaches the screen ▸

One thing to clarify before I write it down. In the design we have, usage preservation comes free: counters live in Redis keyed by tenant and period, independent of tier, and the Lua script just compares against whatever limit it receives. Resetting usage on a tier change would actually be *extra* code.

So I'd interpret your decision like this:

- **In scope:** the tier change takes effect on the next request. Counters are simply left untouched.
- **Out of scope, listed as a preferable enhancement:** specifying and testing the period-usage semantics on tier change. That covers the upgrade-resume and downgrade-over-limit scenarios, how the dashboard shows "used is above the new limit," and so on.

If you meant something else, for example actively resetting usage on change, tell me and I'll adjust.

Question 6: dashboard data and how it reaches the browser.

My proposal:

What the dashboard shows

- **Header:** gateway connection status, quota enforcement status (OK / degraded because Redis is down), and a countdown to the next period reset.
- **Traffic:** allowed and rejected requests per second right now, plus a short trend over the last 60 seconds.
- **Period totals:** allowed, rejected, number of tenants near their limit (≥ 80%), number of tenants over it.

- **Tenants table:** the top N tenants by usage percentage, with tier, used/limit, rejected, a status badge (OK / near / blocked), and a tier dropdown to change it. Plus a search by tenant ID to find any tenant outside the top N.

Where the numbers come from

- The gateway keeps in-memory aggregates: per-second counters in a 60-slot ring buffer, and a map of each tenant's latest used/rejected, updated from the Lua script's return values. So building a dashboard snapshot never scans Redis. Iterating 50,000 keys every second would be the wrong load pattern.
- This relies on the single-instance assumption. In production, each gateway instance would export metrics to a shared pipeline.

How it gets to the browser

- **Server-Sent Events:** the gateway pushes a snapshot (global metrics + top N) once per second. Tier changes and tenant search go through plain REST.
- SSE fits because the data flow is one-way, server to browser, and the browser's `EventSource` reconnects automatically, which helps with the "connection dropped and recovered" state.
- 1-second polling would also work and is simpler. WebSocket would be overkill, since we don't need two-way streaming.

Seeding

- 50,000 tenants generated at startup with a tier mix of roughly 70% Bronze, 25% Silver, 5% Gold. That proves the top-N design at the stated scale. The traffic generator drives a configurable active subset of them.

Does this shape work for you: SSE, top N plus search, and the full 50,000 tenants seeded?

> regarding conters - you got it correctly, they remain untouched on tier change. The new tier will take effect on the next request according to the counter state. The dashboard shape - I agree. SSE - I think polling will be a reasonable simplification for the ptototype. And I don;t think we have to seed all 50K users. The reasonable amount, around 10- 20 should be enough. It must not kill the laptop it is running on. Remember, we'll emulate load

Settling the list view and the local load rate ▸

Recorded:

- **Polling** instead of SSE. The dashboard calls one snapshot endpoint (e.g. `GET /admin/snapshot`) every second. SSE goes into architecture.md as the production direction.

- About 15 seeded tenants with a mix like 8 Bronze, 5 Silver, 2 Gold. With so few, the table just shows all tenants sorted by usage percentage; top N and search become unnecessary. I'll note them as what a 50,000-tenant dashboard would need.
- Load sized for a laptop: the generator's default rate is modest (hundreds of req/s, not 4,200), and the per-period limits are tuned so that rate pushes tenants into near and blocked states within the 1-minute period. The peak numbers stay as production design input in the non-functional requirements, not as a prototype target.

Question 7: unhappy states on the dashboard. With polling these become fairly straightforward. My proposal:

State	Detected by	Dashboard shows
Initial loading	first poll not yet answered	A loading indicator in place of the panels; no zeros pretending to be data
Backend down	poll fails (network error, timeout ~2 s, or 5xx)	A red "Gateway unreachable" banner, last known data greyed out with "last updated N s ago"; tier controls disabled; polling continues
Recovered	next poll succeeds after failures	Banner disappears, briefly shows "Reconnected," data refreshes, controls re-enabled
Enforcement degraded	the snapshot reports Redis unavailable	A yellow "Quota enforcement degraded (fail-open)" banner; traffic numbers still live
Tier change failed	the PUT returns an error or the gateway is down	Inline error on that row; the dropdown reverts to the actual tier

Also: an overlap guard, so a slow poll doesn't stack with the next one, and a stale-data rule. If a poll fails while the backend is merely slow, we keep showing the last data rather than flashing an error at once. For example, the banner appears after 2 consecutive failures.

Does this cover the unhappy states the way you'd like, or should anything be added or simplified?

remain the top tenants idea. Let's just make it top 10. regarding question 7 - disagree here "the snapshot reports Redis unavailable" the napshot report should not know about redis. I guess this has to be a separate monitoring of it's health

Checking the Redis health endpoint options ▸
Agreed on both.

Top 10 tenants by usage percentage in the table. Search by tenant ID becomes an out-of-scope enhancement.

Health is separate from the snapshot. The snapshot carries only business data: traffic, period totals, tenants. Infrastructure health gets its own endpoint, and Spring Boot Actuator gives us that almost for free:

- `GET /actuator/health` includes a built-in Redis health indicator (it comes with the Spring Data Redis starter), reporting `redis: UP/DOWN` as a component, plus the overall gateway status. We configure a short Redis timeout so the health call itself doesn't hang when Redis is gone.
- The dashboard polls health separately from the snapshot (every few seconds is enough) and shows the yellow "Quota enforcement degraded (fail-open)" banner when Redis is down.
- If the gateway itself is unreachable, both polls fail and the red "Gateway unreachable" banner takes precedence.
- In production this same endpoint serves as the liveness and readiness probe target, which fits nicely in architecture.md.

The updated unhappy-state table: initial loading, backend down, recovered, and tier change failed stay as proposed; "enforcement degraded" is driven by the health poll instead of the snapshot.

One small follow-up before we move on. During a Redis outage, requests are let through without being counted. Do you want a traffic metric in the snapshot like "requests allowed without enforcement" (count per second and per period), so the operator can see how much slipped through? It describes traffic, not Redis, so it fits the snapshot's responsibility. But it's optional, and dropping it keeps things simpler.

1. strongly disagree to mention the usage of redis health check as liveness or rediness probe on prod - the redis absence must now stop the API. We can put it for the moment in the health check of the service, but just for simplisity. Ideally it must be monitored separately. 2. we calculate trafic with redis. If it is down we can't calculate enithing, we are just open. In a single node implementation we can still count in memory, but in a cluster we can't. So, let's not fake it in prototype. If redis is down - it;s just down, the dashboard displays it, the trafic remains the same

Reconciling in-memory traffic counters with Redis-based counting ▸

You're right on both, and point 1 was a real mistake on my part. Putting Redis into a readiness probe would let the orchestrator pull every gateway instance out of rotation during a Redis outage, which is exactly the fail-closed behavior we rejected. Recorded:

1. **Redis health sits in the gateway's `/actuator/health` only as a prototype simplification,** so the dashboard has one place to check. architecture.md states explicitly that in production Redis is monitored separately, and that the gateway's liveness/readiness probes must not depend on Redis, since a missing Redis must not stop the API.
2. **No in-memory fallback counting.** When Redis is down the gateway is simply open: requests pass, nothing is counted, and the dashboard says so. No "allowed without enforcement" metric.

Point 2 does conflict with something I proposed earlier. In question 6 I put the dashboard aggregates in the gateway's memory: a per-second ring buffer and a map of tenant usage built from Lua results. Your cluster argument applies to those too. With several gateway instances, each would only see its own share of the traffic. So to be consistent, I'd move the aggregates into Redis as well:

- **The same Lua script** that checks and increments the tenant counter also increments global per-second buckets (`traffic:allowed:{epochSecond}`, `traffic:rejected:{epochSecond}`) with a short TTL of about 2 minutes. Still one atomic round trip per request.
- **The snapshot reads from Redis:** the last 60 per-second buckets, plus each tenant's used/rejected for the current period. It then combines them with the tier limits from the in-memory registry and computes the top 10 by usage percentage in the gateway. With 15 tenants that's a trivial read. For 50,000 tenants, architecture.md notes a sorted set maintained by the script as the production approach.
- **When Redis is down,** the snapshot endpoint returns 503 and the dashboard shows "Metrics unavailable" over the traffic and tenant panels, alongside the degraded banner from the health poll. The gateway is still reachable, so this state is distinct from "Gateway unreachable."

One detail to settle with this: during a Redis outage, should tier changes still work? They only touch the in-memory registry, so technically yes. I'd keep them enabled, with the tenant list shown without usage numbers. Or would you rather the whole tenant panel go to "unavailable"?

> Agree to store the global counter in redis as well. For the snapshot, we can use the in-memory storage. regading snashot - let's talk it through. if it is 50K accounts, make 50K calls to the redis every second seams being a not good idea.

Checking per-tier sorted sets for exact top-tenant ordering ▸

Agreed: global counters in Redis, and the snapshot is served from gateway memory.

That gives the snapshot two layers. A background job in the gateway rebuilds it once per second from Redis and keeps the result in memory. `GET /admin/snapshot` just returns

that cached object. Redis load is then independent of how many dashboards are open: 1 operator or 50 cost the same.

The remaining question is exactly yours: how does that job read tenant usage without touching every tenant? Even batched (one `MGET` of 50,000 keys instead of 50,000 calls), it would move 50,000 values every second just to keep 10 rows. Wrong shape.

Proposal: per-tier sorted sets maintained by the Lua script.

- **The key insight is that within a tier, every tenant has the same limit.** Ordering by usage percentage within a tier is the same as ordering by `used`, so integer counts are enough. No floating-point ratios.
- **On every allowed request** the script also runs `ZINCRBY usage:{tier}:{period} 1 {tenantId}`. That's 3 sorted sets per period (Bronze, Silver, Gold), each with a TTL of 2 periods like the other keys.
- **Top 10 overall:** take the top 10 of each tier with `ZREVRANGE`, which gives at most 30 candidates. Compute the percentage for each against its tier limit, then keep the top 10. A tenant in the true global top 10 is necessarily in its own tier's top 10, so the merge is exact.
- **"Near limit" and "over limit" counts:** `ZCOUNT` per tier with the score ranges `[0.8 × limit, limit)` and `[limit, +inf)`, which is O(log N). No tenant scan.
- **Rejected per tenant:** only needed for the 10 rows shown, so one `MGET` of 10 keys.
- **Global traffic:** the 60 per-second buckets (one `MGET`) plus period totals for allowed and rejected (two keys, incremented in the same script).

All of that fits in one pipelined round trip per second, whether there are 15 tenants or 50,000. We use the same design in the prototype, because it costs no more than the naive version and it's the part that has to hold at scale.

What this adds: tier changes now touch Redis. When a tenant moves from Bronze to Silver, their entry must move from `usage:bronze:{period}` to `usage:silver:{period}`, keeping the score. A second small Lua script does the move atomically (`ZSCORE`, `ZREM`, `ZADD`).

That brings back the question from last time in a sharper form. If Redis is down when an operator changes a tier, the in-memory registry updates, but the move fails. The tenant's usage then sits in the wrong tier's set until the period ends: at most 1 minute in the prototype, up to a day in production.

My suggestion is to **reject tier changes while Redis is down** (503, the inline error on the row we already designed). The registry and the sorted sets then never disagree, and an operator rarely needs to change tiers during an outage. Does this sorted-set design work for you, and do you agree with blocking tier changes during a Redis outage?

ok, top 10 doesn't look to me as a good idea. Let's switch the something else. Let's show any 10 tenants and support the search by name. We'll still display on the screen not more than 10 tenants in a grid, so 10 request to redis is fine. let's still

remain sets of near the limit and over the limit. I guess the global counters will be enough, not by tier. Let's allow change of tier while redis is down, let's not do anything with the possible discrepancy, just mention it as an acceptable issue for prototype

Choosing how near and over-limit tenants are tracked ▸

Here's how I read your decisions; correct me where I got it wrong.

Tenant grid

- Tenants get a name in the registry (seeded like "Acme Corp", "Globex"...).
- With no search, the grid shows 10 tenants in a fixed, deterministic order (alphabetical by name). With a search, it shows up to 10 tenants whose name contains the search text, case-insensitive.
- Both filtering and selection happen on the in-memory registry (fine even at 50,000 entries). Usage for the 10 shown is then one `MGET` of their used/rejected keys.
- Search by name joins the snapshot request (e.g. `GET /admin/snapshot?q=acme`). The cached-snapshot idea then applies only to the global part, while the 10 rows are read per request.

Near and over limit as Redis sets

- The Lua script keeps two sets per period, `near:{period}` and `over:{period}`. On each request it puts the tenant in the right set for its current usage and current limit (≥ 80% → near, ≥ 100% → over).
- The dashboard shows their sizes (`SCARD`), which is O(1). The per-tier sorted sets are gone.

Traffic counters

- Global only, not split by tier: allowed and rejected per second, plus period totals.

Tier changes

- Always allowed, even while Redis is down. Without the per-tier sets, a tier change doesn't touch Redis at all, so the earlier discrepancy disappears.
- What remains is smaller: near/over membership is recalculated only on the tenant's next request. A tenant upgraded while blocked stays in the "over" set until they call again or the period ends. requirements.md lists that as an accepted prototype limitation.

Question 8: the traffic generator. My proposal:

- **A separate small Java service** in docker compose, sending requests to the gateway's business endpoint with `X-Tenant-Id`. It uses Java 21's `HttpClient` with virtual threads, so a few hundred req/s is cheap on a laptop.

- **Skewed, configurable traffic.** Tenants get different weights, so within each 1-minute period some Bronze tenants blow through their limit, a few Silver tenants come close, and Golds stay comfortable. The dashboard then shows all three states. Parameters: total rate, duration, weight profile.
- **Running it:** on demand, e.g. a script like `./load.sh --rps 300 --duration 5m` that starts the generator container, rather than starting with `docker compose up`. A quiet dashboard at startup is a useful state to see too.

The open choice: **controlled from the command line only, or also from the dashboard** (a small panel with start/stop and a rate slider)? Dashboard controls make demos smoother, but they couple the admin UI to a test tool, and the generator would need its own control API. I'd keep it command-line only. What do you prefer?

no, I think it's ok to start it in a docker compose as another node and have the predefined json with tenant descriptions - their names and tier. no need to update it on change tier from dashboard, it's ok to have them the same after restart. The implementation - java is ok, maybe even more lightweight - node.js?

Comparing Node.js and Java for the traffic generator ▸
Recorded:

- **The generator starts with** `docker compose up` as its own container and runs continuously, so the dashboard moves as soon as everything is up.
- **One predefined** `tenants.json` (id, name, tier), mounted into both the gateway and the generator. The gateway seeds its registry from it; the generator uses it to know which tenants to send traffic for. Tier changes from the dashboard live only in gateway memory and reset to the file on restart. The generator never needs to know about them.
- **The traffic shape** lives in the generator's own config: total rate plus per-tenant weights. That keeps `tenants.json` a pure description of tenants. The defaults are skewed so that within each 1-minute period some tenants get blocked, some come near their limit, and some stay well below it.

Node.js for the generator: yes, I'd go with it.

- **No new toolchain.** The frontend already brings Node and npm into the repo through Vite, so this adds nothing, and it can be tested with Vitest like the frontend.
- **It's light.** A Node Alpine image with one script is small, starts instantly, and uses tens of MB of memory, where the JVM would be in the hundreds. The built-in `fetch` with keep-alive on an event loop handles a few hundred req/s easily on a single thread.
- **It's simple.** It's about a hundred lines of TypeScript. Current Node LTS runs `.ts` files directly by stripping the types, so there's no build step.

- **What Java would give us:** sharing types with the gateway. That isn't worth much here, because the generator only needs a URL, a header and a tenant list.

Question 9: the closing list. This is what I'd put into requirements.md as out of scope and assumptions. Tell me what to add or remove, and then I'll write the four files.

Out of scope

- Authentication and authorization, for both API clients (the tenant ID arrives already resolved from an upstream layer) and the admin dashboard
- Proxying to a real upstream (the gateway returns a stub response)
- A separate admin service, and separate Redis monitoring (production direction, described in architecture.md)
- A database or message broker, and persisting tier changes across restarts
- Multiple gateway instances, and Redis clustering or high availability
- Per-tenant timezones, sliding windows, and borrowing from future periods
- Specified usage semantics on tier change, such as upgrade-resume and downgrade-over-limit display (a preferable enhancement)
- Editing tier limits, and an audit log of tier changes
- Pagination, and search beyond a case-insensitive name match
- Push delivery via SSE or WebSocket (polling is used instead)
- Controlling the generator from the dashboard
- Proving the production load of about 4,200 req/s; it's a design input only, with no load testing at that rate
- Visual design, a component library, or a design system

Assumptions

- Single gateway instance, with the tenant registry held in its memory
- Quota period of 1 minute in the prototype, configurable, aligned to UTC epoch time and shared by all tenants (production: 1 day)
- Per-period limits configurable per tier; prototype values tuned so the generator's default rate produces all three states
- The `X-Tenant-Id` header is trusted; a missing header gets 400
- Fail-open when Redis is unavailable; nothing is counted during the outage, and the dashboard shows that metrics are unavailable
- Redis health appears in the gateway's health endpoint only as a prototype simplification
- Near/over set membership is updated only on a tenant's next request after a tier change (an accepted limitation)

Per-tenant timezones, sliding windows, and borrowing from future periods - that was not even requested. just put in requirements the fixed window strategy. these things are not out of scope, they are not needed

Created 4 files, shared files ▸

I removed that line: requirements.md now just states the fixed window strategy in FR-3.

A few things I settled while writing, so you can check them:

- **The snapshot always returns 200 when Redis is down.** The tenant rows still come back with names and tiers, and only the metric parts are null. That reconciles two earlier decisions: "metrics unavailable" during an outage, and "tier changes allowed while Redis is down." The dashboard needs the rows to offer tier changes.
- **An unknown `X-Tenant-Id` returns 400, the same as a missing header.** It's only a defensive guard, since the Lua script needs some limit to compare against.
- **Production notes added to architecture.md.** Redis Cluster would reject our multi-key Lua script, because it mixes per-tenant and global keys. At about 4,200 req/s, the global counters become hot keys and would need sharding.
- **The dashboard is served by nginx** with a reverse proxy to the gateway, so the browser uses one origin and needs no CORS configuration.
- **The backlog has 13 changes in 6 phases,** from `bootstrap-skeleton` to `calibration-and-demo`. The starting values for limits and generator rate are in requirements.md and get tuned in the last item.

Requirements Document · MD	Download
Architecture Document · MD	Download
Backlog Document · MD	Download
Task Document · MD	Download

Download all

let's also do this change - Pagination, and search beyond a case-insensitive name match - let's remain and implement it

Planning pagination and filters through the status sets ▸

Before I update the files, one thing to pin down: what should search cover beyond name? My proposal:

Filters, combinable

- **Name:** contains, case-insensitive (as now).
- **Tenant ID:** exact match, for jumping to one tenant.
- **Tier:** Bronze / Silver / Gold.
- **Status:** ok / near / over. This one is the most useful for the operator ("show me everyone blocked"), and it's cheap with what we have. For near or over, the gateway reads that period's set (`SMEMBERS quota:near:{p}`) and intersects it with the registry. For ok, it takes all tenants minus both sets.

Pagination

- Page size 10, matching the grid. Offset-based (`page` , `size`), sorted by name, and the response includes the total count.
- The dashboard keeps the current page and filters while polling, so the 1-second refresh doesn't jump back to page 1.

Consequences to note

- The status filter inherits the accepted limitation: after a tier change, a tenant's near/over membership is stale until its next request. So it can briefly show up under the wrong status.
- When Redis is down, the status filter is unavailable (disabled in the UI). Name, ID and tier filters still work, since they only need the registry.
- At 50,000 tenants, "over" could hold thousands of IDs. Reading the whole set per poll is still fine, and the production note would be `SSCAN` or a search index.

Is this the search you had in mind: name, ID, tier and status? Or do you want something different or fewer filters?

no, exactly this on, I agree

Ran 2 commands, edited a file, shared files ▸

Pagination and the extended search are now in scope across all three files; task.md is unchanged.

- **requirements.md:**

  - FR-14 defines the combinable filters (name contains, exact ID, tier, status) and offset pagination: sorted by name, page size 10, with a total count.
  - The tenant grid keeps the current page and filters across polls, and the status filter is disabled while metrics are unavailable.
  - The near/over staleness after a tier change now also applies to the status filter.
  - Pagination and search are removed from out of scope; only top-N by usage stays there.
- architecture.md:
  - The snapshot rows flow filters in the registry first, then reads the near/over sets with `SMEMBERS` when filtering by status.
  - At 50,000 tenants, the production notes add cursor pagination, a search index and `SSCAN`.
- backlog.md now has 15 changes. Two are new: `tenant-search-and-pagination` on the gateway and `dashboard-tenant-filters` on the dashboard, so the basic snapshot and grid stay small changes.

One detail I decided while writing: if the dashboard requests a status filter while Redis is down, the gateway marks the rows as unavailable rather than returning an error. The UI disables that filter in this state anyway, so this only covers requests that arrive regardless.

Requirements
Document · MD
Download

Architecture
Document · MD
Download

Backlog
Document · MD
Download

Download all

NFR-2 Per-request cost: the quota decision costs exactly one Redis round trip, independent of the number of tenants. - this is wrong. the decision must cost near 0, so we'll check in memory and update redis counters asynchronously

Weighing trade-offs of local counting with periodic Redis sync ▸

That's a valid design, and at 4,200 req/s it's arguably the better one: Redis drops off the hot path, and the request no longer waits for a network hop. But it reverses several decisions we wrote down, so let's settle them before I edit the files.

What changes

- **The decision becomes local.** The gateway keeps per-tenant counters for the current period in memory and checks them directly, at nanosecond-level cost. A background flusher sends accumulated deltas to Redis every N ms in one pipeline (`INCRBY` for used, rejected and the global counters), and reads back the global totals.
- **FR-6 / NFR-5 (no overshoot) only holds on a single instance.** With several gateway instances, each one decides on the last synced total plus its own unsynced deltas. A tenant can overshoot by roughly `other instances × that tenant's request rate × sync interval`. For example, with 10 instances, a 200 ms sync and a tenant at 500 req/s, that's up to about 90 extra requests at the boundary. That's noise for Gold, but large for a Bronze limit of a few hundred per day.
- **D7 (no in-memory counting) is reversed.** That was your cluster argument. In-memory counting now *is* the design, and Redis becomes the aggregation point between instances and the source for the dashboard.
- **Redis-down behavior changes.** The gateway keeps enforcing from its local counters instead of being fully open. That's exact on one node and approximate across many, since instances stop seeing each other's traffic. The "nothing is counted during an outage" assumption goes away, and fail-open shrinks to "enforcement degrades to per-instance."
- **Restarts and cold start.** In-memory state is lost on restart, so the gateway must load current-period counters from Redis, either on startup or lazily on a tenant's first request. Otherwise a restarted instance would give every tenant a fresh quota.
- **Near/over sets and per-second metrics** move into the flusher instead of the Lua script, which itself becomes a batch "apply deltas" script.

The question: which consistency model do we commit to?

- **A. Local counters + periodic sync, bounded overshoot accepted.** It's simple and fits the prototype well: on our single instance it's exact. In production, overshoot is bounded by the sync interval and documented as an accepted trade-off.
- **B. Quota leasing.** Each instance reserves a slice of a tenant's remaining quota from Redis (e.g. an atomic "take up to 50") and decides locally until the slice runs out. There's no overshoot even across instances, and Redis is touched once per slice, not once per request. The costs: more logic, quota stranded in unused slices near the end of a period, and small limits like Bronze need small slices, which brings back Redis calls.

I'd go with A for the prototype and describe B in architecture.md as the option for tenants that need strict limits. A or B? Then we'll settle the Redis-down behavior and the sync interval.

no, not this way. the instance holds in memory a set of only tenants, that esceed the limit. the request comes, if it is not there, it pass. and then asynchronously in a

different thread it dumps this fact to the redis, updating counter, and the near/over limit buckets. on response from redis it decides - is the tenant allready over limit, then put him inot the set, or not. It reduces the precision by several calls, but it is acceptable. Plus the redis failure becomes simple in this case - the gateway just continue to work with whatever it has in memory

Listing the failure modes and their trade-offs ▸

Understood, that's a much lighter model than the ones I proposed. Here's how I'd write it down; check that it matches.

Request path (no Redis)

1. Each gateway instance holds an in-memory **blocked set** for the current period: the tenants known to be over limit.
2. A request comes in. If the tenant is in the set → 429 with `Retry-After`. Otherwise → allowed and served.
3. Either way, the request thread enqueues an event (tenant, allowed/rejected, second) and returns. Cost: a set lookup and a queue offer.

Async path (a separate thread)

4. The thread sends events to Redis. A Lua script increments `used` or `rejected`, the global per-second and period counters, and updates the near/over sets against the tenant's current limit. For allowed events it returns `used`.
5. If `used ≥ limit`, the tenant goes into the local blocked set.

What follows from it

- **Precision:** a tenant overshoots by the requests that arrive between crossing the limit and the Redis response, roughly `rate × async delay`. So `used` in Redis can end up slightly above the limit. FR-6 and NFR-5 become "bounded overshoot, accepted."
- **Multiple instances work without extra machinery.** The Redis count is global, so every instance blocks a tenant on its own next response. The overshoot is per instance, with the same bound. D7 stays true: counting itself still happens in Redis.
- **Period boundary:** the blocked set belongs to a period index and is dropped when the period changes. Everyone starts fresh, matching the fixed window.
- **Tier change:** the tenant is removed from the local blocked set. Its next request is re-evaluated in Redis against the new limit, which re-blocks it if it's still over. That handles both upgrade and downgrade. (With separate instances in production, the others learn on their next event too.)
- **Restart:** the set starts empty, so each blocked tenant gets a few extra requests before being re-blocked. Accepted. Optionally, a warm-up could load `quota:over:{p}` on startup.
- **Redis down:** the gateway keeps enforcing from the blocked set it has. Tenants already blocked stay blocked until the period ends; no new ones get blocked; nothing is counted. The dashboard behavior we designed stays the same.

- **NFR-2 becomes:** the quota decision is an in-memory lookup with no I/O. Redis is off the request path.

The remaining question: the event queue between request threads and the Redis thread.

My proposal:

- A **bounded queue.** When it's full (Redis slow or down), new events are **dropped** and a dropped-events counter is logged. Request threads never block, and memory can't grow without limit. During an outage the counts are lost anyway, so dropping costs nothing extra.
- The thread **drains the queue in batches** (e.g. everything available every ~50 ms) and aggregates per tenant and per second before calling Redis. At 4,200 req/s that turns thousands of increments into a few hundred tenant deltas per batch. The script then applies `INCRBY` deltas and still returns `used` per tenant.

The trade-off: batching adds up to one batch interval to the async delay, so overshoot grows slightly. Sending event by event is simpler but costs one Redis operation per request. Batch with aggregation, or one event at a time for the prototype?

I think update the redis on one request is absolutely right. Plus it will be working correctly more or less in multy nodes

Ran 5 commands, shared files ▸

The three files now describe the in-memory blocked set with one asynchronous Redis call per request.

- **requirements.md:**
  - FR-4 to FR-6 define the in-memory decision and the async recording, with bounded overshoot accepted.
  - FR-6a: the blocked set is discarded at the period boundary, and each event carries its own period and second.
  - FR-6b: the event queue is bounded and drops events on overflow.
  - FR-8 now says the gateway keeps working with its blocked set when Redis fails. FR-11: a tier change removes the tenant from the blocked set.
  - NFR-2, NFR-4 and NFR-5 are rewritten for the new model.
  - The scenarios, assumptions (including the restart behavior) and the degraded-banner text are updated to match.
- **architecture.md:**
  - The `QuotaService` part is replaced by `BlockedTenants` and `UsageRecorder`.
  - There's a new request-flow sequence and record script with no check in it, plus notes on overshoot and Redis failure.

  - Decisions D5–D7 are rewritten, with D7a added for the bounded queue.
  - The production notes now cover restart warm-up, tier changes across instances and clock skew. Batching is mentioned only as a fallback if Redis load ever becomes a limit.
- **backlog.md:** Phase 2 is now four changes: `async-usage-recording`, `blocked-set-enforcement`, `redis-failure-and-health` and `traffic-metrics-recording`. The tier-change item now covers removing the tenant from the blocked set. That makes 16 changes in total.

One detail I added: the writer only adds a tenant to the blocked set if the event's period is still the current one. Otherwise, a late response for the old period could block a tenant at the start of a fresh period.

Requirements
Document · MD
Download

Architecture
Document · MD
Download

Backlog
Document · MD
Download

Download all

architecture, point 2.3 - "Plain React state and hooks; no state management or UI library. The 60-second trend is a hand-written SVG sparkline." - explain in details in the chat what it is about?

Weighing plain state and hooks against Redux for this app
That line in 2.3 makes three choices about how the dashboard is built.

1. "Plain React state and hooks": no Redux or similar

The dashboard's state is small and almost all of it comes from the server:

- the latest snapshot (traffic, totals, tenant rows);
- the latest health result;
- connection status: loading, OK, failed N times, reconnected;
- what the operator controls: filters, the current page, a pending tier change and its error.

Every second the poll replaces the snapshot entirely. Nothing is edited on the client and merged back. So there's no need for a global store like Redux, Redux Toolkit or Zustand, with actions, reducers and selectors. Those pay off when many distant components read and modify shared client-side state, and that isn't the case here.

Instead, the state lives in a few places:

- `useState` / `useReducer` in the top-level `App` component, or in a small hook, holds the data.
- **Custom hooks** encapsulate the logic. For example, `usePolling` does the interval, the timeout, the no-overlap guard and counting consecutive failures. `useSnapshot(filters, page)` and `useHealth()` are built on it.
- **Props** pass the data down to the panels: header, traffic, totals, tenant grid. The tree is shallow, so prop passing stays simple.

A sketch of the core hook:

ts

```ts
function usePolling<T>(url: string, intervalMs: number) {
  const [data, setData] = useState<T | null>(null);
  const [failures, setFailures] = useState(0);
  useEffect(() => {
    let inFlight = false;
    const tick = async () => {
      if (inFlight) return;                        // no overlapping polls
      inFlight = true;
      try {
        const res = await fetch(url, { signal: AbortSignal.timeout(2000) });
        if (!res.ok) throw new Error(String(res.status));
        setData(await res.json());
        setFailures(0);
      } catch { setFailures(f => f + 1); }         // 2+ → "Gateway unreachable"
      finally { inFlight = false; }
    };
    tick();
    const id = setInterval(tick, intervalMs);
    return () => clearInterval(id);
  }, [url, intervalMs]);
  return { data, failures };
}
```

The unhappy states fall out of these values. `data === null` means initial loading. `failures ⩾ 2` means unreachable, with the old `data` still there to show greyed out.

`failures` dropping back to 0 after that means "Reconnected." Since the URL includes the filters and page, changing a filter restarts polling with the new query.

There's one honest alternative: **TanStack Query** is built for exactly this kind of server state. It has `refetchInterval`, retries, caching and `isLoading` / `isError` flags. It would be a perfectly reasonable choice. I left it out to keep the dependency count low and the unhappy-state logic explicit and testable, but if you'd rather use it, it fits better than Redux would.

2. "No UI library"

The dashboard doesn't use component kits like MUI, Ant Design or Chakra. It's built from plain HTML elements, such as `<table>`, `<select>`, `<input>` and `<button>`, plus one small CSS file for layout, the red and yellow banners, the status colors and the greyed-out stale state. The task explicitly says a component library isn't needed, only a readable screen.

3. "A hand-written SVG sparkline"

A sparkline is a tiny line chart with no axes, grid or legend. It shows the shape of the trend, not exact values. Ours draws the last 60 seconds of allowed and rejected req/s, which the snapshot already returns as two arrays of 60 numbers.

Instead of pulling in a chart library like Recharts or Chart.js for one small chart, we draw it directly. React renders SVG natively, so the component just converts the numbers into points of a `<polyline>`:

tsx

```tsx
function Sparkline({ allowed, rejected, width = 300, height = 60 }:
  { allowed: number[]; rejected: number[]; width?: number; height?: number }) {
  const max = Math.max(1, ...allowed, ...rejected);
  const points = (values: number[]) =>
    values.map((v, i) =>
      `${(i / (values.length - 1)) * width},${height - (v / max) * height}`
    ).join(" ");
  return (
    <svg width={width} height={height} role="img" aria-label="Requests per second,
      <polyline points={points(allowed)} fill="none" stroke="green" />
      <polyline points={points(rejected)} fill="none" stroke="red" />
    </svg>
  );
}
```

Each value becomes an (x, y) point. X spreads the 60 seconds evenly across the width. Y scales the value against the maximum in the window, flipped because SVG's y axis points

down. The current req/s numbers are shown as text next to it, so the exact values are still readable.

Why this beats a library here:

- It's about 20 lines with no dependency.
- It re-renders every second without the entry animations that many chart libraries play by default, which look jittery when data updates that often.
- It's easy to test with React Testing Library by checking the rendered `points` attribute.

if we don't use redux here - won't it be a huge state propagation over whole tree from the app level?

Weighing re-render cost against React.memo and state splitting ▸

No. Prop drilling becomes a problem when data has to pass through many intermediate components that don't use it themselves. This tree is too shallow for that:

```
App                      ← useSnapshot(filters, page), useHealth()
├── StatusBanner         ← connection + health
├── Header               ← period, countdown, statuses
├── TrafficPanel         ← traffic series, current req/s
│   └── Sparkline
├── TotalsPanel          ← period totals, near/over counts
└── TenantGrid           ← rows, total, filters, page, disabled flag
    ├── TenantFilters
    ├── TenantRow × 10   ← one row + onTierChange
    └── Pager
```

Each panel gets exactly the slice it needs, one or two levels down. The deepest path is App → TenantGrid → TenantRow, and TenantGrid actually uses what it passes on. There's no chain of components that just hand data through.

Re-rendering

The other half of the concern is re-rendering. Every second the snapshot is replaced, so `App` re-renders, and with it all the panels. But Redux wouldn't save much here. The selectors in Redux skip re-rendering components whose slice didn't change, and in our case almost everything changes every second: traffic, totals, the countdown, usage in the rows. And the whole screen is a few dozen elements. Re-rendering it once per second costs a fraction of a millisecond.

If it ever became noticeable, there are cheap targeted fixes before reaching for a store:

- `React.memo` on `TenantRow`, so rows whose data didn't change skip re-rendering. That's useful if the page size ever grows.
- **Keeping filter typing local:** the search input keeps its own state and commits to the query with a short debounce, so typing doesn't re-render the whole app on every keystroke.
- **Context for cross-cutting values,** like the "gateway unreachable" flag that both the banner and the tier selectors need. If that flag ever had to reach deeper components, a small `ConnectionContext` would remove the drilling without a global store.

When Redux (or Zustand) would be the right call

- The tree is deep, and many unrelated components read the same data.
- The client owns and edits significant state: optimistic updates, undo, drafts, merging changes back.
- Several features act on the same data from different places, and you want one predictable update path with devtools and time travel.

None of that applies to this dashboard. It's a read-mostly view of server data plus a handful of controls. That's why the choice is plain hooks, with TanStack Query as the alternative if we wanted a library: it targets this kind of server state, which Redux doesn't.

add to the repository layout the scripts folder, there will be scripts to shut down and start components individually. and the docs folder, for these initial files

Ran a command, shared files ▸

The repository layout in architecture.md now has `/scripts` and `/docs`, and the start script moved from the root into `/scripts`.

- `/scripts` holds scripts to start and stop the whole stack or each component individually: Redis, gateway, dashboard, generator. The Technology table now points there too.
- `/docs` holds task.md, requirements.md, architecture.md and backlog.md. Their links to each other are relative, so they keep working once moved there.

Two places changed to match:

- **requirements.md:** a new FR-28 for starting and stopping components individually.
- **backlog.md:**
  - `bootstrap-skeleton` now creates the scripts and moves the initial documents into `/docs`.
  - `calibration-and-demo` uses the component scripts for the outage demos.

Architecture

Document · MD

Download

Requirements
Document · MD

Download

Backlog
Document · MD

Download

Download all